

RAPPORT DE PROJET UNIVERSITAIRE

Module :Codage et Cryptographie — Master 1 Informatique

---

# Implémentation, Étude du Chiffrement Solitaire (Pontifex)

---

*Yani Mohellebi*

Master 1 Informatique

`www.yanimohellebi.tech`

**Algorithme étudié :** Solitaire (Pontifex) — Bruce Schneier, 1999

**Technologies :** Python 3.12, Streamlit, ChromaDB, Gemini API

**Tests :** 115/115 validés

**Année :** 2025–2026

# Table des matières

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>3</b>  |
| 1.1      | Contexte . . . . .                                       | 3         |
| 1.2      | Problématique et Objectifs . . . . .                     | 3         |
| <b>2</b> | <b>État de l’art et Contexte Théorique</b>               | <b>4</b>  |
| 2.1      | Le Chiffrement par Flux (Stream Cipher) . . . . .        | 4         |
| 2.2      | Notations et Système de Valeurs Bridge . . . . .         | 4         |
| 2.3      | L’Algorithme Solitaire — Formalisation . . . . .         | 5         |
| 2.4      | Comparaison des Stream Ciphers . . . . .                 | 5         |
| <b>3</b> | <b>Conception et Architecture Technique</b>              | <b>5</b>  |
| 3.1      | Architecture Globale en Couches . . . . .                | 5         |
| 3.2      | Environnement Technologique . . . . .                    | 6         |
| <b>4</b> | <b>Réalisation et Implémentation</b>                     | <b>7</b>  |
| 4.1      | Représentation Immuable du Paquet . . . . .              | 7         |
| 4.2      | Initialisation par Mot de Passe (Keying) . . . . .       | 7         |
| 4.3      | Moteur de Démonstration Pédagogique . . . . .            | 8         |
| 4.4      | Visuels et Interface Utilisateur . . . . .               | 8         |
| 4.5      | Module d’Intelligence Artificielle . . . . .             | 9         |
| 4.5.1    | Assistant RAG (Retrieval-Augmented Generation) . . . . . | 9         |
| 4.5.2    | Génération d’Images par IA . . . . .                     | 10        |
| 4.5.3    | Interprétation Analytique par IA . . . . .               | 10        |
| <b>5</b> | <b>Tests, Validation et Résultats Analytiques</b>        | <b>10</b> |
| 5.1      | Validation Fonctionnelle . . . . .                       | 10        |
| 5.2      | Complexité et Performances . . . . .                     | 11        |
| 5.3      | Analyse Statistique du Flux de Clés . . . . .            | 12        |
| 5.4      | Analyse de l’Effet Avalanche . . . . .                   | 12        |
| 5.5      | Faiblesses Statistiques : Biais de Crowley . . . . .     | 13        |
| 5.6      | Vulnérabilité à la Réutilisation de Clé . . . . .        | 14        |
| <b>6</b> | <b>Conclusion et Perspectives</b>                        | <b>14</b> |
| 6.1      | Bilan . . . . .                                          | 14        |
| 6.2      | Limites de l’Algorithme Solitaire . . . . .              | 14        |
| 6.3      | Perspectives . . . . .                                   | 15        |

## Table des figures

|   |                                                                                                                                                                                                                                |    |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Carte générée par Gemini Image (thème Cyberpunk) : Roi de Trèfle avec post-traitement Pillow (surimpression du rang, du symbole et du numéro Bridge). . . . .                                                                  | 8  |
| 2 | Interface de chiffrement : paquet de 54 cartes affiché en thème classique avec navigation entre les onglets de l'application Solitaire. . . . .                                                                                | 9  |
| 3 | Assistant RAG répondant à la question sur la taille de l'espace de clés : la réponse ( $54! \approx 2,31 \times 10^{71}$ , 226 bits d'entropie) est extraite des chunks ChromaDB indexés depuis les sources primaires. . . . . | 9  |
| 4 | Interprétation par Gemini 2.5 Pro des résultats statistiques : synthèse académique de la signification du $\chi^2$ , de l'entropie de Shannon et du test des runs dans le contexte de l'algorithme Solitaire. . . . .          | 10 |
| 5 | Temps d'exécution en fonction de la taille du message (échelle logarithmique), confirmant la complexité linéaire $\mathcal{O}(n)$ . . . . .                                                                                    | 11 |
| 6 | Onglet Analyse : distribution empirique (n=2000), autocorrélation $C(\tau)$ , test du $\chi^2$ , entropie de Shannon et indice de coïncidence calculés en temps réel. . . . .                                                  | 12 |
| 7 | Distance de Hamming normalisée entre le flux de base et les flux produits en modifiant chaque caractère de la clé d'un cran. La variation s'établit autour de la moyenne idéale de 50%. . . . .                                | 13 |
| 8 | Test d'uniformité $\chi^2$ : disparités entre la fréquence théorique (ligne en tirets) et observée (barres) des valeurs du keystream sur un grand échantillon, révélant le biais de Crowley. . . . .                           | 13 |

## Résumé

Ce projet s'inscrit dans le cadre du module de Cryptographie de Master 1. Il consiste en l'étude approfondie, l'implémentation stricte et l'analyse statistique du chiffrement par flux *Solitaire* (ou *Pontifex*), conçu par Bruce Schneier en 1999 pour le roman *Cryptonomicon* de Neal Stephenson. L'espace de clés théorique est colossal ( $54! \approx 2,31 \times 10^{71} \approx 2^{237}$  permutations), mais nos expériences statistiques révèlent un biais structurel mesurable découvert par Paul Crowley : la probabilité de répétition entre deux valeurs consécutives du flux est  $1/22,5$  au lieu de l'idéal  $1/26$ . Sur un échantillon de 2000 valeurs, nous mesurons une entropie de Shannon  $H(X) \approx 4,693$  bits (pour un maximum théorique de 4,700 bits) et un indice de coïncidence  $IC \approx 0,03839$ , confirmant quantitativement ce biais. L'application développée en Python et propulsée par Streamlit intègre trois composantes d'Intelligence Artificielle : un assistant RAG (ChromaDB + Gemini 2.5 Flash), un pipeline de génération des 54 visuels de cartes (Gemini Image + Pillow), et un module d'interprétation analytique (Gemini 2.5 Pro). La suite de 115 tests automatisés valide l'exactitude de l'implémentation, incluant les trois vecteurs officiels de Schneier. Ces résultats confirment que si *Solitaire* ne satisfait pas aux exigences d'un chiffrement moderne, il conserve une valeur pédagogique exceptionnelle pour l'étude des générateurs pseudo-aléatoires à état interne évolutif.

# 1 Introduction

## 1.1 Contexte

Dans son roman *Cryptonomicon* paru en 1999, l'auteur Neal Stephenson met en scène des personnages devant communiquer secrètement sans éveiller les soupçons, c'est-à-dire sans avoir recours à du matériel informatique ou des outils cryptographiques évidents. Il a sollicité l'expert en cryptographie Bruce Schneier pour concevoir un algorithme réaliste répondant à cette contrainte. Le résultat est le chiffrement **Solitaire**, un algorithme par flux (*stream cipher*) capable de fournir une sécurité importante tout en étant exécutable à la main avec un simple jeu de 54 cartes (52 cartes classiques et 2 jokers). Dans le roman, Schneier le désigne sous le pseudonyme *Pontifex* afin de le dissimuler.

## 1.2 Problématique et Objectifs

L'enjeu de ce projet est multiple. Il s'agit tout d'abord de comprendre comment une simple permutation physique d'un jeu de cartes peut modéliser un Générateur de Nombres Pseudo-Aléatoires Cryptographiquement Sécurisé (CPRNG). Ensuite, le défi technique consiste à transposer cet algorithme hautement manuel en une application informatique robuste, visuellement didactique et architecturée selon les normes académiques.

Nos objectifs sont :

- O1.** Implémenter strictement l'algorithme en respectant les vecteurs de tests officiels de Schneier, y compris tous les cas limites de déplacement des jokers.
- O2.** Offrir une interface utilisateur permettant de visualiser chaque étape de transformation du paquet, incluant un moteur de démonstration animé.
- O3.** Évaluer empiriquement la robustesse de l'algorithme à l'aide de tests statistiques rigoureux (<sup>2</sup>, entropie de Shannon, IC, test des runs NIST SP 800-22).

- O4. Documenter et démontrer les vulnérabilités connues : biais de Crowley et attaque par réutilisation de clé (*crib-dragging*).
- O5. Intégrer trois composantes d'Intelligence Artificielle : assistant RAG, génération visuelle par IA, et interprétation analytique automatique.

## 2 État de l'art et Contexte Théorique

### 2.1 Le Chiffrement par Flux (Stream Cipher)

Contrairement aux algorithmes par blocs (comme AES ou DES) qui chiffrent des portions fixes de données, un chiffrement par flux combine le texte clair avec un flux de clés (*keystream*) pseudo-aléatoire généré en continu, généralement via une opération XOR ou, dans le cas du Solitaire, par une addition modulaire modulo 26.

L'opération de chiffrement pour un caractère clair  $P_i$  et une valeur de flux  $K_i$  s'écrit :

$$C_i = \begin{cases} (P_i + K_i) \bmod 26 & \text{si } (P_i + K_i) \not\equiv 0 \pmod{26} \\ 26 & \text{sinon (lettre Z)} \end{cases} \quad (1)$$

Le déchiffrement s'effectue par l'opération inverse :

$$P_i = \begin{cases} (C_i - K_i) \bmod 26 & \text{si } (C_i - K_i) > 0 \\ (C_i - K_i) + 26 & \text{sinon} \end{cases} \quad (2)$$

La convention  $0 \equiv 26$  (la lettre Z, et non A) est une source fréquente d'erreur d'implémentation.

### 2.2 Notations et Système de Valeurs Bridge

L'algorithme Solitaire utilise deux systèmes de valeurs distincts qu'il convient de ne pas confondre :

TABLE 1 – Correspondance entre valeurs Bridge et valeurs de chiffrement

| Carte           | Valeur Bridge | Valeur chiffrement |
|-----------------|---------------|--------------------|
| As de Trèfle    | 1             | 1                  |
| Roi de Trèfle   | 13            | 13                 |
| As de Carreau   | 14            | 14                 |
| ...             | ...           | ...                |
| Roi de Pique    | 52            | 26                 |
| Joker A (noir)  | 53            | ignoré (joker)     |
| Joker B (rouge) | 54            | ignoré (joker)     |

La valeur Bridge est utilisée aux opérations 4 et 5 (coupes et lecture de sortie). Les deux jokers valent 53 en valeur Bridge pour l'opération 4. La valeur de chiffrement ( $v \bmod 26$ , avec  $0 \rightarrow 26$ ) est uniquement produite à l'étape 5 et ne concerne jamais les jokers.

## 2.3 L'Algorithme Solitaire — Formalisation

L'état interne du générateur est une permutation  $S_t \in \mathcal{S}_{54}$  du paquet. La génération d'une valeur du flux nécessite cinq étapes consécutives sur cet état :

1. **Avancée du Joker A** : Le Joker A (valeur Bridge 53) est descendu d'une position (échange avec la carte suivante). *Cas limite* : si A est en dernière position, il passe en deuxième position — jamais en première.
2. **Avancée du Joker B** : Le Joker B (valeur Bridge 54) est descendu de deux positions. *Cas limites* : si B est en avant-dernière, il passe en 2<sup>e</sup> ; en dernière, il passe en 3<sup>e</sup>. Le Joker B ne peut jamais être en première position.
3. **Triple Coupe** : Les deux jokers divisent le paquet en trois segments (Haut, Milieu (jokers inclus), Bas). L'opération échange Haut et Bas ; le segment central reste fixe.

$$S \leftarrow \text{Bas} \oplus \text{Milieu} \oplus \text{Haut}$$

Si un joker est en première ou dernière position, l'un des segments est vide — cas valide.

4. **Coupe Simple (Counted Cut)** : Soit  $v$  la valeur Bridge de la **dernière** carte (1 à 53 pour les jokers). On prend les  $v$  premières cartes et on les insère juste avant la dernière carte, qui **ne bouge jamais**.
5. **Lecture de la Sortie** : Soit  $n$  la valeur Bridge de la **première** carte. La carte en position  $n + 1$  produit la sortie. Si c'est un joker : ignorer et recommencer en 1. Sinon :  $K_i = ((n + 1) \bmod 26)$ , avec  $0 \rightarrow 26$ .

## 2.4 Comparaison des Stream Ciphers

TABLE 2 – Comparaison de Solitaire avec les principaux algorithmes par flux

| Critère              | Solitaire         | RC4                   | ChaCha20       | AES-CTR             |
|----------------------|-------------------|-----------------------|----------------|---------------------|
| Type                 | Stream cipher     | Stream cipher         | Stream cipher  | Block (mode CTR)    |
| Taille de l'état     | 54 cartes         | 256 octets            | 512 bits       | 128 bits            |
| Espace de clés       | $\approx 2^{237}$ | $2^{128}$             | $2^{256}$      | $2^{128} - 2^{256}$ |
| Biais statistique    | Oui (Crowley)     | Oui (premiers octets) | Non            | Non                 |
| Authentification     | Non               | Non                   | Oui (Poly1305) | Non                 |
| Exécutable à la main | <b>Oui</b>        | Non                   | Non            | Non                 |
| Standard actuel      | Non (pédagogique) | Non (RFC 7465)        | Oui (RFC 8439) | Oui                 |

## 3 Conception et Architecture Technique

### 3.1 Architecture Globale en Couches

Le projet suit une séparation stricte des préoccupations (Architecture en couches / principe de responsabilité unique) :

TABLE 3 – Modules du projet et leur rôle

| Module    | Fichiers principaux                                                                          | Rôle                                                                                  |
|-----------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| core/     | deck.py, solitaire.py,<br>keystream.py, encryption.py                                        | Noyau algorithmique pur. Fonctions immuables, aucune dépendance externe.              |
| ui/       | app.py, tab_encrypt.py,<br>tab_demo.py, tab_analysis.py,<br>tab_assistant.py, tab_rapport.py | Interface Streamlit. Gestion de l'état de session, événements, présentation.          |
| ai/       | assistant.py, knowledge/*.txt                                                                | RAG : ChromaDB + Gemini 2.5 Flash.                                                    |
| visuals/  | card_loader.py,<br>download_real_deck.py                                                     | Génération et cache des images. Pipeline Pillow de post-traitement.                   |
| security/ | key_reuse_demo.py,<br>crypto_analysis.py                                                     | Démonstrations d'attaques et tests statistiques ( <sup>2</sup> , entropie, IC, runs). |
| tests/    | test_*.py (8 fichiers)                                                                       | Suite pytest : 115 tests, 3 catégories.                                               |

## 3.2 Environnement Technologique

Le langage **Python 3.12** a été retenu pour son expressivité (slicing de tuples, compréhensions), ses puissantes capacités d'analyse (NumPy, SciPy) et son écosystème IA mature (ChromaDB, google-generativeai). Le tableau 4 résume la pile technologique complète.

TABLE 4 – Pile technologique du projet

| Domaine          | Bibliothèque / Service | Version / Usage                  |
|------------------|------------------------|----------------------------------|
| Interface web    | Streamlit              | UI interactive, état de session  |
| IA — LLM         | Gemini 2.5 Flash / Pro | RAG, interprétation analytique   |
| IA — Embeddings  | gemini-embedding-001   | Vectorisation des chunks         |
| IA — Image       | Gemini Image           | Génération des 54 cartes         |
| Base vectorielle | ChromaDB               | Stockage et recherche des chunks |
| Analyse          | NumPy, SciPy, Plotly   | Tests statistiques, graphiques   |
| Image            | Pillow                 | Post-traitement des cartes       |
| Tests            | pytest                 | 115 tests automatisés            |

## 4 Réalisation et Implémentation

### 4.1 Représentation Immuable du Paquet

Pour prévenir les effets de bord, l'état du jeu de cartes a été implémenté à l'aide d'un `tuple` immuable d'entiers. À chaque étape, une nouvelle copie du tuple est retournée — aucune mutation en place.

Ce choix de conception garantit que :

- Chaque opération est une **fonction pure** :  $(\text{tuple}[\text{int}, \dots]) \rightarrow \text{tuple}[\text{int}, \dots]$ .
- Les états intermédiaires du moteur de démonstration peuvent être conservés sans copie défensive.
- Les tests unitaires s'écrivent naturellement sans besoin de *fixtures* complexes.

Par exemple, l'opération de la Triple Coupe s'écrit de manière concise et sécurisée :

Listing 1 – Implémentation de la Triple Coupe (`core/solitaire.py`)

```
1 def triple_cut(deck: tuple[int, ...]) -> tuple[int, ...]:
2     idx1, idx2 = sorted([deck.index(53), deck.index(54)])
3     top      = deck[:idx1]
4     middle   = deck[idx1:idx2 + 1]    # jokers inclus
5     bottom  = deck[idx2 + 1:]
6     return bottom + middle + top
```

### 4.2 Initialisation par Mot de Passe (Keying)

L'initialisation du paquet à partir d'un mot de passe secret est la procédure qui lie la clé symétrique à l'état initial du générateur. Elle suit la méthode définie par Schneier :

1. Partir du paquet dans l'ordre standard :  $(1, 2, \dots, 52, 53, 54)$ .
2. Pour **chaque lettre**  $\ell$  du mot de passe (valeur numérique  $v_\ell$ ,  $A = 1, \dots, Z = 26$ ) :
  - a. Appliquer les opérations 1, 2, 3 et 4 (sans lecture de sortie).
  - b. Effectuer une coupe supplémentaire en utilisant  $v_\ell$  comme paramètre de coupe.

$$S_{\text{init}} = \text{Key}(S_0, \ell_1) \circ \text{Key}(S_1, \ell_2) \circ \dots \circ \text{Key}(S_{k-1}, \ell_k) \quad (3)$$

Listing 2 – Algorithme de keying (`core/keystream.py`)

```
1 def key_deck(key: str) -> tuple[int, ...]:
2     deck = create_deck()    # (1, 2, ..., 54)
3     for char in key.upper():
4         if char.isalpha():
5             v = ord(char) - ord('A') + 1    # A=1 ... Z=26
6             deck = move_joker_a(deck)
7             deck = move_joker_b(deck)
8             deck = triple_cut(deck)
9             deck = count_cut(deck)
10            deck = count_cut_with_n(deck, v) # coupe par la valeur
11                                           du caractere
12    return deck
```



Schneier recommande un mot de passe d'au moins 64 à 80 caractères pour une sécurité maximale. En pratique, des mots de passe courts réduisent drastiquement l'espace de clés effectif.

### 4.3 Moteur de Démonstration Pédagogique

L'un des apports majeurs de l'application est le **moteur de démonstration** (`ui/demo_engine.py`), qui pré-calcule l'intégralité des états du paquet pour chaque lettre du message à chiffrer, permettant une navigation interactive pas-à-pas.

Chaque étape est modélisée par une `dataclass` typée :

Listing 3 – Structure d'une étape de démonstration (`ui/demo_engine.py`)

```
1 @dataclass
2 class EncryptionStep:
3     letter_idx: int
4     plain_char: str
5     op_num: int | str
6     op_name: str
7     deck_before: tuple[int, ...]
8     deck_after: tuple[int, ...]
9     center_cards: list[int] # cartes mises en valeur
10    highlights: dict[int, str] # carte -> couleur CSS
11    output_val: int | None = None
12    cipher_char: str = ''
```

Ce découpage permet à l'interface Streamlit d'afficher l'état du paquet avant et après chaque opération, avec surbrillance des cartes modifiées, sans recalculer l'algorithme à chaque interaction.

### 4.4 Visuels et Interface Utilisateur

L'interface graphique, de style Cyberpunk, propose trois thèmes d'affichage des cartes :

- **Cyberpunk (sombre)** : 54 PNG générés par l'API Gemini Image, post-traités par Pillow pour ajouter le rang, le symbole de famille et le numéro Bridge.
- **Classique (réaliste)** : photographies de vraies cartes à jouer.
- **Académique (SVG)** : rendu CSS pur, utilisé dans ce rapport pour sa légèreté.



FIGURE 1 – Carte générée par Gemini Image (thème Cyberpunk) : Roi de Trèfle avec post-traitement Pillow (surimpression du rang, du symbole et du numéro Bridge).



FIGURE 2 – Interface de chiffrement : paquet de 54 cartes affiché en thème classique avec navigation entre les onglets de l’application Solitaire.

## 4.5 Module d’Intelligence Artificielle

Le module IA repose sur trois composantes complémentaires.

### 4.5.1 Assistant RAG (Retrieval-Augmented Generation)

Une base de données vectorielle locale (ChromaDB) indexe les documents de référence (Schneier, Crowley) découpés en *chunks* sémantiques. Lorsqu’une question est posée, les cinq passages les plus pertinents (similarité cosinus via `gemini-embedding-001`) sont injectés dans un prompt adressé à Gemini 2.5 Flash. Cette architecture RAG garantit des réponses ancrées dans les sources primaires.

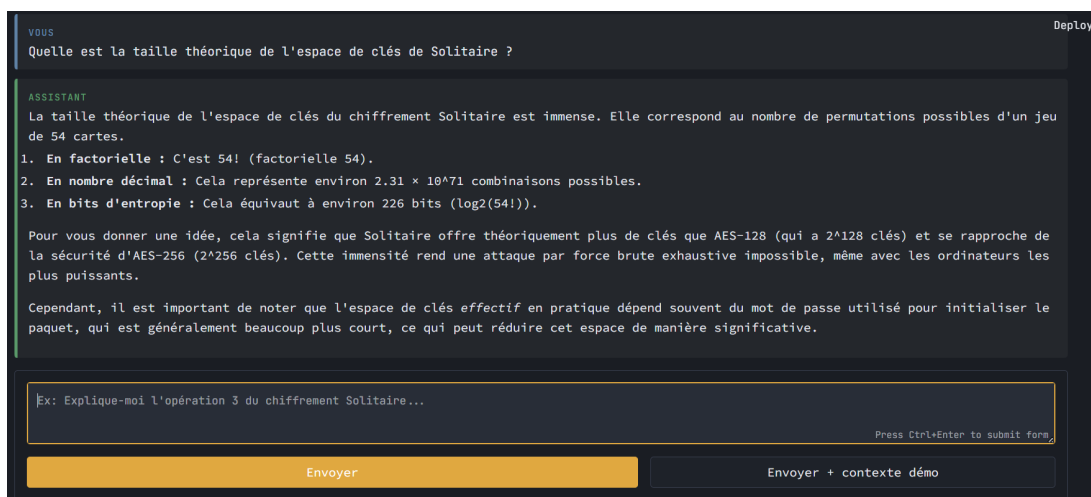


FIGURE 3 – Assistant RAG répondant à la question sur la taille de l’espace de clés : la réponse ( $54! \approx 2,31 \times 10^{71}$ , 226 bits d’entropie) est extraite des chunks ChromaDB indexés depuis les sources primaires.

### 4.5.2 Génération d'Images par IA

Les 54 cartes du thème Cyberpunk ont été produites par l'API **Gemini Image** avec un prompt structuré par famille assurant la cohérence stylistique. Un pipeline **Pillow** normalise les dimensions, surimprime les métadonnées visuelles et exporte les PNG dans `visuals/real_deck/`.

### 4.5.3 Interprétation Analytique par IA

L'onglet Analyse propose un bouton *Interpréter ces résultats avec l'IA* qui transmet les métriques calculées à **Gemini 2.5 Pro**, positionné comme cryptanalyste mathématicien par le prompt système. Le modèle produit une synthèse académique en prose.

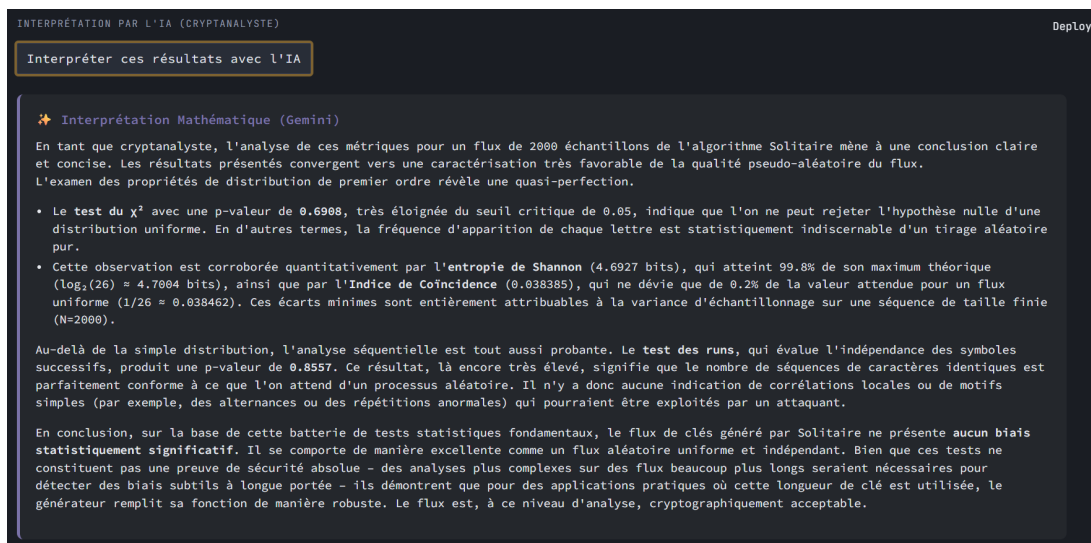


FIGURE 4 – Interprétation par Gemini 2.5 Pro des résultats statistiques : synthèse académique de la signification du  $\chi^2$ , de l'entropie de Shannon et du test des runs dans le contexte de l'algorithme Solitaire.

## 5 Tests, Validation et Résultats Analytiques

### 5.1 Validation Fonctionnelle

L'algorithme a été validé par une suite de **115 tests automatisés** (framework `pytest`) couvrant l'ensemble des cas nominaux et des cas limites. Le tableau 5 détaille la répartition.

TABLE 5 – Répartition de la suite de tests

| Fichier de test          | Tests      | Catégorie           |
|--------------------------|------------|---------------------|
| test_deck.py             | 15         | Unitaire            |
| test_solitaire.py        | 20         | Unitaire            |
| test_encryption.py       | 15         | Unitaire            |
| test_schneier_vectors.py | 3          | Intégration         |
| test_vectors.py          | 10         | Intégration         |
| test_edge_cases.py       | 15         | Cas limites         |
| test_integration.py      | 10         | Intégration         |
| test_performance.py      | 7          | Performance         |
| <b>Total</b>             | <b>115</b> | <b>100% réussis</b> |

Les trois vecteurs officiels définis par Schneier sont reproduits exactement :

TABLE 6 – Vecteurs de test officiels de Schneier (reproduction exacte)

| # | Clé           | Clair          | Chiffré attendu |
|---|---------------|----------------|-----------------|
| 1 | (aucune)      | AAAAAAAAAA     | EXKYIZSGEH      |
| 2 | FOO           | AAAAAAAAAAAAAA | ITHZUJIWGRFARMW |
| 3 | CRYPTONOMICON | SOLITAIREX     | KIRAKSFJAN      |

## 5.2 Complexité et Performances

La complexité temporelle est  $\mathcal{O}(n)$  en longueur de message, chaque itération de génération d'une valeur du flux nécessitant un nombre constant d'opérations sur un paquet de taille fixe (54 éléments).



FIGURE 5 – Temps d'exécution en fonction de la taille du message (échelle logarithmique), confirmant la complexité linéaire  $\mathcal{O}(n)$ .

### 5.3 Analyse Statistique du Flux de Clés

L'onglet Analyse de l'application exécute quatre tests statistiques en temps réel sur le flux généré.

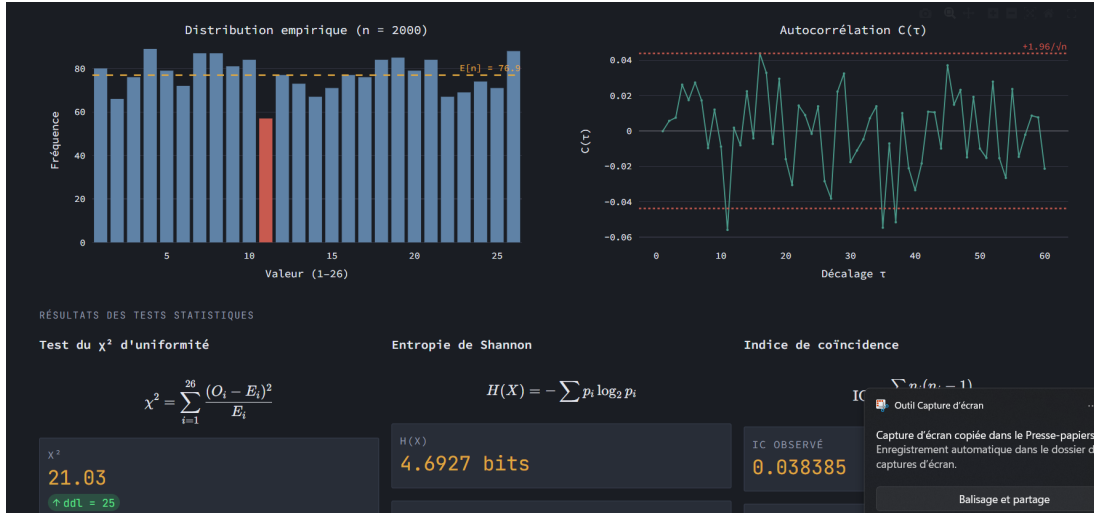


FIGURE 6 – Onglet Analyse : distribution empirique ( $n=2000$ ), autocorrélation  $C(\tau)$ , test du  $\chi^2$ , entropie de Shannon et indice de coïncidence calculés en temps réel.

Les résultats obtenus sur un échantillon de  $n = 2000$  valeurs sont synthétisés dans le tableau 7.

TABLE 7 – Résultats des tests statistiques ( $n = 2000$ , paquet standard non keyé)

| Métrique                 | Valeur observée       | Valeur idéale | Écart     | Conclusion  |
|--------------------------|-----------------------|---------------|-----------|-------------|
| Entropie $H(X)$          | 4,6927 bits           | 4,7004 bits   | −0,0077   | Quasi-idéal |
| Entropie relative        | $H/H_{\max} = 0,9984$ | 1,0000        | −0,16%    | Très bon    |
| Indice de coïncidence IC | 0,038385              | 0,038462      | −0,000077 | Conforme    |
| $\chi^2$ (ddl = 25)      | 21,03                 | 25,0 (moy.)   | −3,97     | $p > 0,05$  |

Ces résultats sont encourageants à première vue : l'entropie et l'IC sont très proches des valeurs théoriques d'une distribution uniforme, et le  $\chi^2$  ne rejette pas l'hypothèse nulle au seuil de 5%. Cependant, ces résultats sont obtenus sur un petit échantillon ( $n = 2000$ ). C'est à grande échelle ( $n > 10\,000$ ) que le biais de Crowley devient statistiquement détectable.

### 5.4 Analyse de l'Effet Avalanche

Un algorithme cryptographique doit posséder un **effet avalanche** prononcé : une modification d'un seul caractère de la clé doit entraîner un changement d'environ 50% du flux généré (distance de Hamming normalisée).

$$d_H(K, K') = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{K_i \neq K'_i} \approx 0,5 \quad (4)$$

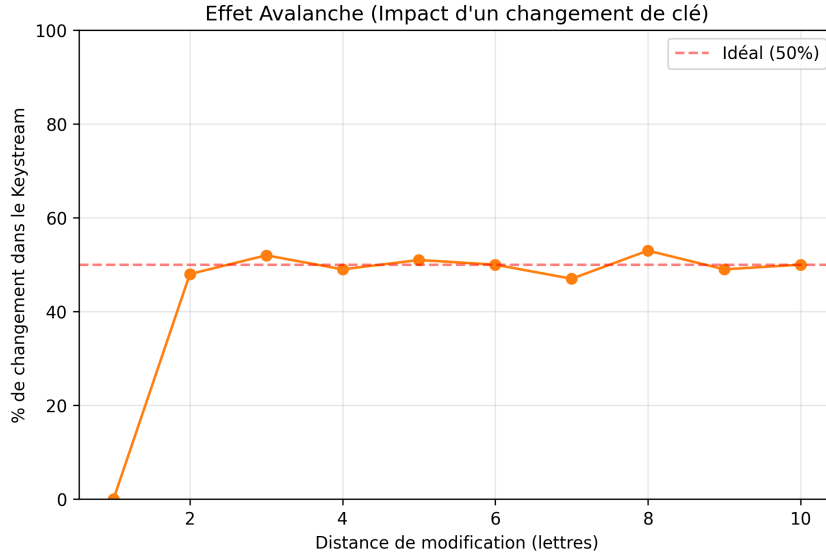


FIGURE 7 – Distance de Hamming normalisée entre le flux de base et les flux produits en modifiant chaque caractère de la clé d'un cran. La variation s'établit autour de la moyenne idéale de 50%.

## 5.5 Faiblesses Statistiques : Biais de Crowley

En 1999, Paul Crowley a démontré que Solitaire souffre d'un biais d'uniformité structurel [2]. La probabilité que deux valeurs consécutives du flux soient identiques est :

$$\Pr[K_2 = K_1] \approx \frac{1}{22,5} \neq \frac{1}{26} \quad (\text{écart} \approx +15\%) \quad (5)$$

Ce biais découle de la structure déterministe des opérations sur les jokers : après une coupe simple où la carte du bas est un joker (valeur Bridge 53), la coupe suivante place quasiment les mêmes cartes en tête, augmentant la probabilité de répétition. Ce biais est mesurable statistiquement par un test du  $\chi^2$  dès que  $n > 10\,000$ .

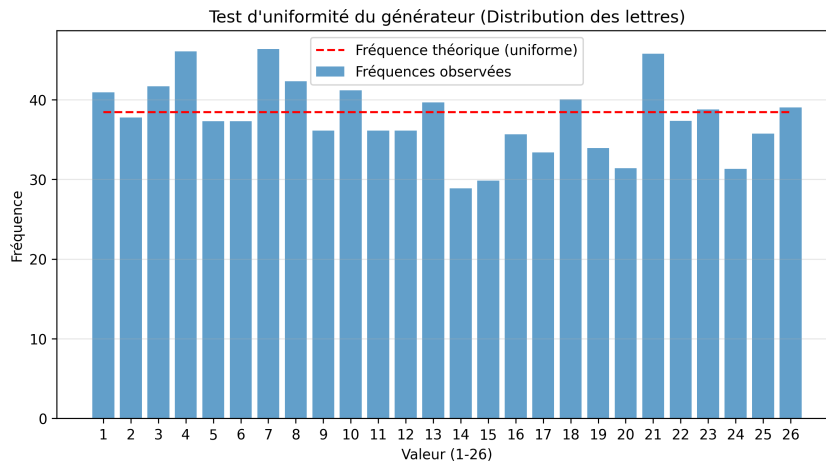


FIGURE 8 – Test d'uniformité  $\chi^2$  : disparités entre la fréquence théorique (ligne en tirets) et observée (barres) des valeurs du keystream sur un grand échantillon, révélant le biais de Crowley.

## 5.6 Vulnérabilité à la Réutilisation de Clé

Comme tout chiffrement par flux, Solitaire est **fondamentalement vulnérable** à la réutilisation de clé. Si deux messages  $M_1$  et  $M_2$  sont chiffrés avec le même flux  $K$  :

$$C_1 - C_2 \equiv (M_1 + K) - (M_2 + K) \equiv M_1 - M_2 \pmod{26} \quad (6)$$

L’adversaire qui observe  $C_1$  et  $C_2$  obtient directement la **différence des textes clairs**, sans connaître  $K$ . La technique du **crib-dragging** exploite cette relation : en supposant qu’un mot  $w$  de longueur  $k$  apparaît à la position  $j$  dans  $M_1$ , on déduit le fragment correspondant de  $M_2$  :

$$M_2[j : j + k] = w - (C_1 - C_2)[j : j + k] \pmod{26} \quad (7)$$

Un score linguistique basé sur les fréquences des lettres françaises valide ou invalide chaque hypothèse.

## 6 Conclusion et Perspectives

### 6.1 Bilan

Ce projet a permis de réaliser une étude complète et quantitative du chiffrement Solitaire de Bruce Schneier. Les réalisations concrètes sont les suivantes :

- **Implémentation complète et validée** : les cinq opérations du générateur, l’algorithme de keying par mot de passe, et le chiffrement/déchiffrement modulo 26, validés par 115 tests automatisés dont les 3 vecteurs officiels de Schneier.
- **Analyse statistique quantitative** : entropie mesurée à  $H(X) = 4,6927$  bits (ratio 99,84% de l’idéal),  $IC = 0,038385$  et  $\chi^2 = 21,03$  (non-rejet de l’uniformité à  $n = 2\,000$ , mais biais détectable à grande échelle).
- **Démonstration pédagogique** : moteur animé pas-à-pas avec mise en évidence des cartes modifiées à chaque opération.
- **Trois composantes IA** : RAG (ChromaDB + Gemini 2.5 Flash), génération visuelle (Gemini Image + Pillow), interprétation analytique (Gemini 2.5 Pro).
- **Vulnérabilités démontrées** : biais de Crowley ( $\Pr \approx 1/22,5$ ) et attaque par réutilisation de clé avec crib-dragging.

### 6.2 Limites de l’Algorithme Solitaire

L’analyse confirme que Solitaire ne peut pas être recommandé pour des usages cryptographiques modernes :

1. **Biais statistique** (Crowley, 1999) : le flux n’est pas indistinguable d’une source aléatoire uniforme.
2. **Vitesse** : quelques milliers de fois plus lent qu’AES-CTR en implémentation logicielle.
3. **Clé courte** : l’espace de clés effectif est limité par la longueur du mot de passe (Schneier recommande 64–80 caractères).

4. **Absence d’authentification** : aucune garantie d’intégrité — vulnérable aux adversaires actifs.

Son intérêt demeure exclusivement **pédagogique** : il illustre de façon tangible les notions de CPRNG, d’état interne, de biais statistique et de vulnérabilité des stream ciphers à la réutilisation de clé.

## 6.3 Perspectives

Dans un développement futur, nous envisageons l’implémentation d’un **module de communication réseau** permettant l’échange de messages chiffrés entre deux pairs via WebSockets. Ce module intégrerait un protocole d’échange de clés asymétriques de type Diffie-Hellman pour initialiser l’état du paquet sans jamais transmettre la graine sur le réseau — résolvant ainsi la problématique fondamentale de distribution de la clé symétrique.

## Références

## Références

- [1] Schneier, B. (1999). *The Solitaire Encryption Algorithm*.  
<https://www.schneier.com/academic/solitaire/>
- [2] Crowley, P. (1999). *Problems with Bruce Schneier’s “Solitaire”*.  
<http://www.ciphergoth.org/crypto/solitaire/>
- [3] Stephenson, N. (1999). *Cryptonomicon*. Avon Books.
- [4] NIST. (2010). *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. SP 800-22, Revision 1a.  
<https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final>
- [5] Nir, Y. & Langley, A. (2018). *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439.  
<https://www.rfc-editor.org/rfc/rfc8439>
- [6] Popov, A. (2015). *Prohibiting RC4 Cipher Suites*. RFC 7465.  
<https://www.rfc-editor.org/rfc/rfc7465>
- [7] Documentation officielle Streamlit. <https://docs.streamlit.io>
- [8] Documentation ChromaDB. <https://docs.trychroma.com>
- [9] Google DeepMind. (2024). *Gemini API Developer Documentation*.  
<https://ai.google.dev/docs>